

Here are the detailed solutions in both English and Chinese, based on the mathematical concepts provided in the document.

以下是基于文档中提供的数学概念所做的中英双语详细解答。

Exercise 1: Spark Parallelism, Partition Sizing, and Amdahl's Law / 练习 1: Spark 并行度、分区大小与阿姆达尔定律

Background / 背景信息:

- **EN:** A Spark batch job processes a $T = 500$ GB dataset stored in $P_0 = 4,000$ HDFS blocks (128 MB each). The cluster has $E = 20$ executors, each with $k = 4$ cores and $m = 16$ GB of memory.
- **ZH:** 一个 Spark 批处理作业处理 $T = 500$ GB 的数据集, 存储为 $P_0 = 4,000$ 个 HDFS 块 (每块 128 MB)。集群配置了 $E = 20$ 个执行器, 每个具有 $k = 4$ 个核心和 $m = 16$ GB 内存。

Part A: Parallelism and Partitioning / A 部分: 并行度与分区

(a) Total number of parallel slots S / 计算总的并行槽数 S :

- **EN:** Based on the formula $S = E \cdot k$. $S = 20 \cdot 4 = 80$ parallel slots.
- **ZH:** 根据公式 $S = E \cdot k$ 。 $S = 20 \cdot 4 = 80$ 个并发槽。

(b) Waves required to process $P_0 = 4,000$ partitions / 处理 $P_0 = 4,000$ 个分区所需的波 (waves) 数:

- **EN:** The number of waves is $\lceil P/S \rceil$. Using $P_0 = 4,000$, the waves = $\lceil 4000/80 \rceil = 50$ waves.
- **ZH:** 根据公式, 波数 = $\lceil P/S \rceil$ 。使用 $P_0 = 4,000$, 波数 = $\lceil 4000/80 \rceil = 50$ 波。

(c) Total stage time T_{stage} / 计算总阶段时间 T_{stage} :

- **EN:** $T_{stage} = \lceil P/S \rceil \cdot t_{task}$. Given $t_{task} = 6$ s, $T_{stage} = 50 \text{ waves} \cdot 6 \text{ s} = 300$ seconds.
- **ZH:** $T_{stage} = \lceil P/S \rceil \cdot t_{task}$ 。已知 $t_{task} = 6$ 秒, $T_{stage} = 50 \text{ 波} \cdot 6 \text{ 秒} = 300$ 秒。

(d) Coalescing down to $P_1 = 80$ partitions / 合并为 $P_1 = 80$ 个分区:

- **EN:** Waves required = $\lceil 80/80 \rceil = 1$ wave. Assuming linear scaling, the new task time is $t'_{task} = 6 \cdot (P_0/P_1)$, so $t'_{task} = 6 \cdot (4000/80) = 300$ seconds. The new stage time $T_{stage} = 1 \text{ wave} \cdot 300 \text{ s} = 300$ seconds.
- **ZH:** 所需波数 = $\lceil 80/80 \rceil = 1$ 波。假设线性扩展, 新任务时间 $t'_{task} = 6 \cdot (P_0/P_1)$, 即

$t'_{task} = 6 \cdot (4000/80) = 300$ 秒。新的阶段时间 $T_{stage} = 1 \text{ 波} \cdot 300 \text{ 秒} = 300$ 秒。

(e) Is coalesce helpful and any concerns? / 合并是否有帮助及潜在隐患:

- **EN:** Coalescing does not speed up processing since both approaches take 300 seconds total. Original data per task was $500 \text{ GB}/4000 = 125 \text{ MB}$. New data per task is $500 \text{ GB}/80 = 6.25 \text{ GB}$. The concern is memory: each executor has 16 GB of memory and 4 cores, meaning each core gets an average of 4 GB. A task needing to process 6.25 GB will exceed its memory budget, likely causing disk spills or Out-of-Memory (OOM) errors.
- **ZH:** 合并并没有加速处理, 因为两种方式的总时间都是 300 秒。原分区每任务数据量为 $500 \text{ GB}/4000 = 125 \text{ MB}$ 。新分区每任务数据量为 $500 \text{ GB}/80 = 6.25 \text{ GB}$ 。隐患在于内存: 每个执行器有 16 GB 内存和 4 个核心, 意味着每个核心仅有 4 GB 内存。每个任务处理 6.25 GB 数据超出了可用内存预算, 极易导致磁盘溢写或内存不足 (OOM) 异常。

Part B: Amdahl's Law / B 部分: 阿姆达尔定律

(f) Speed-up for $f = 0.90$ and $S = 80$ / 计算 $f = 0.90$ 且 $S = 80$ 时的加速比:

- **EN:** Using Amdahl's Law $Speed - up(S) = \frac{1}{(1-f)+f/S}$. With $f = 0.90$ and $S = 80$, $Speed - up(80) = \frac{1}{0.10+0.90/80} = \frac{1}{0.10+0.01125} \approx 8.99x$.
- **ZH:** 使用阿姆达尔定律 $Speed - up(S) = \frac{1}{(1-f)+f/S}$ 。已知 $f = 0.90$ 且 $S = 80$, $Speed - up(80) = \frac{1}{0.10+0.90/80} = \frac{1}{0.10+0.01125} \approx 8.99$ 倍。

(g) Theoretical maximum speed-up as $S \rightarrow \infty$ / $S \rightarrow \infty$ 时的理论最大加速比:

- **EN:** The max speed-up is bounded by $1/(1 - f)$. For $f = 0.90$, this is $1/0.10 = 10x$.
- **ZH:** 最大加速比的上限为 $1/(1 - f)$ 。对于 $f = 0.90$, 最大加速比为 $1/0.10 = 10$ 倍。

(h) New speed-up with $S' = 160$ and relative improvement / 扩大至 $S' = 160$ 后的新加速比及相对提升:

- **EN:** For $S' = 160$, $Speed - up(160) = \frac{1}{0.10+0.90/160} = \frac{1}{0.10+0.005625} \approx 9.47x$. The relative improvement is $9.47/8.99 \approx 1.053$, or a ~5.3% improvement.
- **ZH:** 针对 $S' = 160$, $Speed - up(160) = \frac{1}{0.10+0.90/160} = \frac{1}{0.10+0.005625} \approx 9.47$ 倍。相对提升为 $9.47/8.99 \approx 1.053$, 即仅提升了约 5.3%。

(i) What f value yields exactly a 1.5x improvement? / 什么样的 f 值能带来正好 1.5 倍的加速提升?

- **EN:** Set $Speed - up(160) = 1.5 \cdot Speed - up(80)$. $\frac{1}{(1-f)+f/160} = 1.5 \cdot \frac{1}{(1-f)+f/80}$. Let $x = 1 - f$. Then $x + f/80 = 1.5x + 1.5f/160$. Solving gives $f/160 = 1 - f \implies 161f = 160 \implies f \approx 0.9938$ (99.38% parallelisable).

-

ZH: 列方程 $Speed - up(160) = 1.5 \cdot Speed - up(80)$ 。 $\frac{1}{(1-f)+f/160} = 1.5 \cdot \frac{1}{(1-f)+f/80}$ 。 设 $x = 1 - f$ 。 可得 $x + f/80 = 1.5x + 1.5f/160$ 。 化简得到 $f/160 = 1 - f \implies 161f = 160 \implies f \approx 0.9938$ (即需要 99.38% 是可并行的)。

Exercise 2: Windowing, Watermarks, and Cost Trade-offs / 练习 2: 窗口化、水位线与成本权衡

Background / 背景信息:

-

EN: $K = 50,000$ distinct devices. Sliding window length $w = 10$ min, slide $s = 2$ min, watermark threshold $\theta = 4$ min.

-

ZH: 处理 $K = 50,000$ 个独立设备。滑动窗口长度 $w = 10$ 分钟, 滑动间隔 $s = 2$ 分钟, 水位线阈值 $\theta = 4$ 分钟。

Part A: Window arithmetic / A 部分: 窗口运算

(a) How many overlapping windows does each event belong to? / 每个事件属于多少个重叠的窗口?

-

EN: Multiplicity $k = w/s$. $k = 10/2 = 5$ overlapping windows.

-

ZH: 根据窗口多重性公式 $k = w/s$ 。 $k = 10/2 = 5$ 个重叠窗口。

(b) How many open windows does Spark maintain? / Spark 维持多少个开放窗口?

-

EN: $N_{open} = \lceil (w + \theta)/s \rceil$. $N_{open} = \lceil (10 + 4)/2 \rceil = 7$ open windows.

-

ZH: $N_{open} = \lceil (w + \theta)/s \rceil$ 。 $N_{open} = \lceil (10 + 4)/2 \rceil = 7$ 个开放窗口。

(c) Total state memory required / 估计所需的总状态内存消耗:

-

EN: 24 bytes per aggregate. Total memory scales as $N_{open} \cdot K \cdot \text{size}$. $7 \cdot 50,000 \cdot 24 \text{ bytes} = 8.4 \text{ MB}$.

-

ZH: 每个聚合为 24 字节。总内存规模等同于 $N_{open} \cdot K \cdot \text{单条大小}$ 。 $7 \cdot 50,000 \cdot 24 \text{ 字节} = 8.4 \text{ MB}$ 。

(d) Switching to a tumbling 10-minute window / 切换为 10 分钟的滚动窗口:

-

EN: For a tumbling window, $s = w = 10$. $N_{open} = \lceil (10 + 4)/10 \rceil = 2$. New memory = $2 \cdot 50,000 \cdot 24 \text{ bytes} = 2.4 \text{ MB}$. It is smaller by a factor of $8.4/2.4 = 3.5x$.

-

ZH: 对于滚动窗口, $s = w = 10$ 。 $N_{open} = \lceil (10 + 4)/10 \rceil = 2$ 个。新内存

$= 2 \cdot 50,000 \cdot 24 \text{ 字节} = 2.4 \text{ MB}$ 。缩小倍数为 $8.4/2.4 = 3.5$ 倍。

Part B: Watermarks and late events / B 部分：水位线与延迟事件

EN Background: Event lateness D is exponential with mean $\mathbb{E}[D] = 45$ seconds. **ZH Background:** 事件延迟 D 服从均值 $\mathbb{E}[D] = 45$ 秒的指数分布。

(e) Rate parameter μ / 计算率参数 μ :

- **EN:** $\mu = 1/\mathbb{E}[D]$. $\mu = 1/45 \approx 0.0222 \text{ s}^{-1}$.
- **ZH:** $\mu = 1/\mathbb{E}[D]$ 。 $\mu = 1/45 \approx 0.0222 \text{ 秒}^{-1}$ 。

(f) Drop probability with $\theta = 4 \text{ min}$ / $\theta = 4$ 分钟 时的丢弃概率:

- **EN:** With $\theta = 4 \text{ min} = 240 \text{ s}$, $\mathbb{P}(\text{drop}) = e^{-\mu\theta}$. $\mathbb{P}(\text{drop}) = e^{-240/45} = e^{-5.333} \approx 0.0048$ (or 0.48%).
- **ZH:** 当 $\theta = 4 \text{ 分钟} = 240 \text{ 秒}$ 时, $\mathbb{P}(\text{drop}) = e^{-\mu\theta}$ 。 $\mathbb{P}(\text{drop}) = e^{-240/45} = e^{-5.333} \approx 0.0048$ (即 0.48%)。

(g) Smallest watermark threshold θ^* for $\alpha = 10^{-4}$ / 丢弃率目标 $\alpha = 10^{-4}$ 对应的最小水位线 θ^* :

- **EN:** $\theta^* = (1/\mu) \cdot \ln(1/\alpha)$. Given $\alpha = 10^{-4}$, $\theta^* = 45 \cdot \ln(10^4) \approx 414.46$ seconds. In minutes, $414.46/60 \approx 6.91$ minutes.
- **ZH:** $\theta^* = (1/\mu) \cdot \ln(1/\alpha)$ 。已知 $\alpha = 10^{-4}$, $\theta^* = 45 \cdot \ln(10^4) \approx 414.46$ 秒。换算为分钟: $414.46/60 \approx 6.91$ 分钟。

(h) Change in state memory required / 状态内存需求的变化:

- **EN:** Using the new $\theta^* = 6.91 \text{ min}$ with original sliding window $w = 10$, $s = 2$:
 $N_{\text{open}} = \lceil (10 + 6.91)/2 \rceil = 9$ windows. New memory = $9 \cdot 50,000 \cdot 24 = 10.8 \text{ MB}$. This is an increase of $10.8 - 8.4 = 2.4 \text{ MB}$.
- **ZH:** 使用新的 $\theta^* = 6.91 \text{ 分钟}$ 以及原滑动窗口参数 $w = 10$, $s = 2$:
 $N_{\text{open}} = \lceil (10 + 6.91)/2 \rceil = 9$ 个窗口。新内存 = $9 \cdot 50,000 \cdot 24 = 10.8 \text{ MB}$ 。这代表内存增加了 $10.8 - 8.4 = 2.4 \text{ MB}$ 。

Part C: Batch vs. streaming cost model / C 部分：批处理 vs. 流处理成本模型

EN Background: Batch cost $C_b = \$8$ per run, streaming cost $r_s = \$0.20$ per minute. **ZH Background:** 批处理每次运行成本 $C_b = \$8$, 流处理每分钟存活成本 $r_s = \$0.20$ 。

(i) Refresh interval T^* where streaming becomes cheaper / 流处理开始比批处理便宜的刷新间隔 T^* :

-

EN: Streaming is cheaper when $T \leq C_b/r_s$. $T^* = 8/0.20 = 40$ minutes.

-

ZH: 当 $T \leq C_b/r_s$ 时流处理更便宜, $T^* = 8/0.20 = 40$ 分钟。

(j) Cheaper option for a 5-minute maximum staleness / 延迟不超过 5 分钟要求下哪种方案更便宜:

-

EN: Staleness is $T = 5$ min. Since $5 \leq 40$, streaming is cheaper. Streaming cost per hour = $60 \cdot \$0.20 = \$12/\text{hr}$. Batch cost per hour = $(60/5) \cdot \$8 = \$96/\text{hr}$. Streaming is cheaper by $\$96 - \$12 = \$84/\text{hour}$.

-

ZH: 延迟要求 $T = 5$ 分钟。因为 $5 \leq 40$, 因此流处理更便宜。流处理每小时成本 = $60 \cdot \$0.20 = \$12/\text{小时}$ 。批处理每小时成本 = $(60/5) \cdot \$8 = \$96/\text{小时}$ 。流处理更便宜, 每小时可节省 $\$96 - \$12 = \$84$ 。